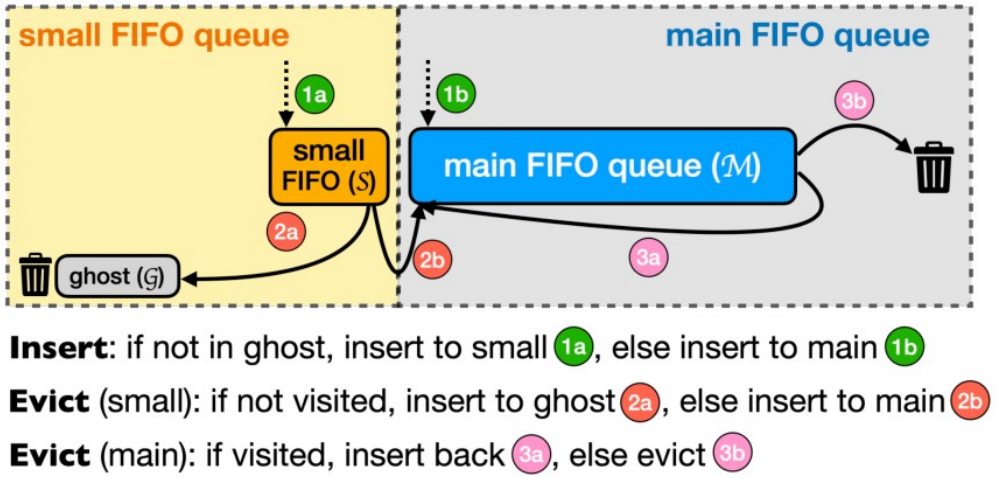
OPERATING SYSTEMS · PAPER PRESENTATION

S3-FIFO: Cache Eviction

FIFO Queues Are All You Need for Cache Eviction

Li Zhiqing (1240016612)
Xia Youran (1240010751)
Zhang Xiaocheng (1240021715)

SOSP 2023 · Juncheng Yang et al.



Original Figure 5 · S3-FIFO architecture
Source: Yang et al., SOSP 2023

Team Structure & Timing

Three speakers follow the supplied script: background, algorithm, then evaluation.

Speaker	Main responsibility
Xia Youran	Background, cache eviction, FIFO / LRU, one-hit wonders.
Li Zhiqing	Three FIFO queues, cache requests and eviction rules.
Zhang Xiaocheng	Experimental setup, results, limitations and conclusion.

10-minute theoretical presentation · Results are reported by the original paper's authors.

PART 1 · MEMBER 1

Background & Motivation

Why does a cache need a better eviction policy?

Background → FIFO vs LRU → One-hit Wonders

What Is Cache Eviction?

Cache space is fast but limited; the system must decide what stays and what goes.

Small & fast cache

A cache is a small and fast storage space that keeps data we may need again.

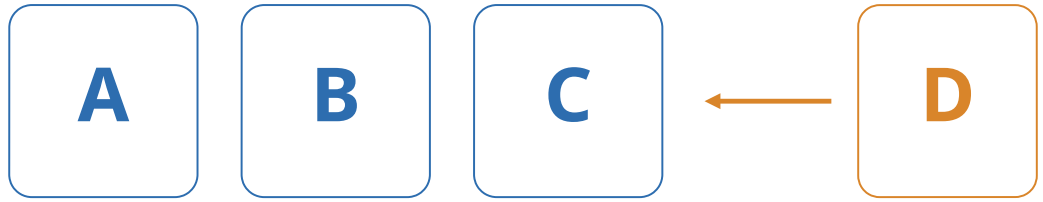
Limited space

When the cache is full, the system must decide which data to remove.

Cache eviction

This process is called cache eviction.

CACHE: ALL SLOTS OCCUPIED



Which object should leave?

Key question: what should stay in the cache, and what should go?

Source: Yang et al., SOSP 2023, §§1–2.

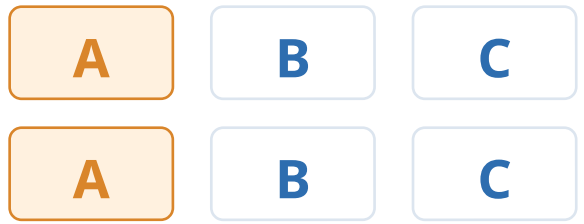
Traditional Algorithms: FIFO vs LRU

Eviction rule · Hit updates · Concurrency cost

FIFO

First In, First Out

Cache hit: A | Left = next eviction

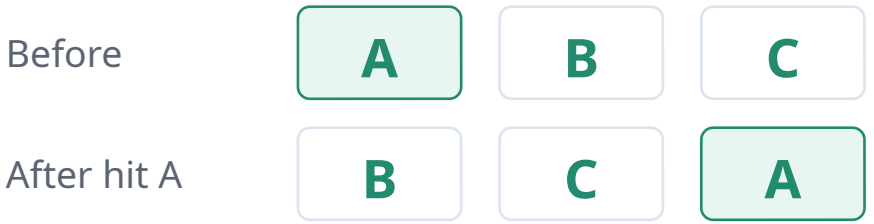


- Evict earliest inserted
- Simple and fast
- May evict useful data

LRU

Least Recently Used

Cache hit: A | Left = next eviction



- Evict least recently used
- Reorder on cache hits
- Overhead with many threads

Goal: FIFO simplicity + better cache efficiency.

Source: Yang et al., SOSP 2023, §2.2.

One-Hit Wonders & Quick Demotion

Shorter access sequences contain a higher fraction of objects with no reuse.

One-hit wonders

Many objects are used only once before they leave the cache: one-hit wonders.

Wasted cache space

They take up cache space without being used again.

Quick demotion

Quick demotion removes less useful objects early.

A SHORTER SEQUENCE → A HIGHER RATIO



start time	end time	sequence length (# objects)	# one-hit wonder	one-hit-wonder ratio
1	17	5	1 (E)	20%
1	7	4	2 (C, D)	50%
1	4	3	2 (B, C)	67%

Original Figure 1 · Source: Yang et al., SOSP 2023

Can we combine the simplicity of FIFO with better cache efficiency?

Source: original Figure 1 and §3, Yang et al., SOSP 2023.

PART 2 · MEMBER 2

How S3-FIFO Works

Thank you. Now, I will explain how S3-FIFO works.

The main idea is simple: three FIFO queues.

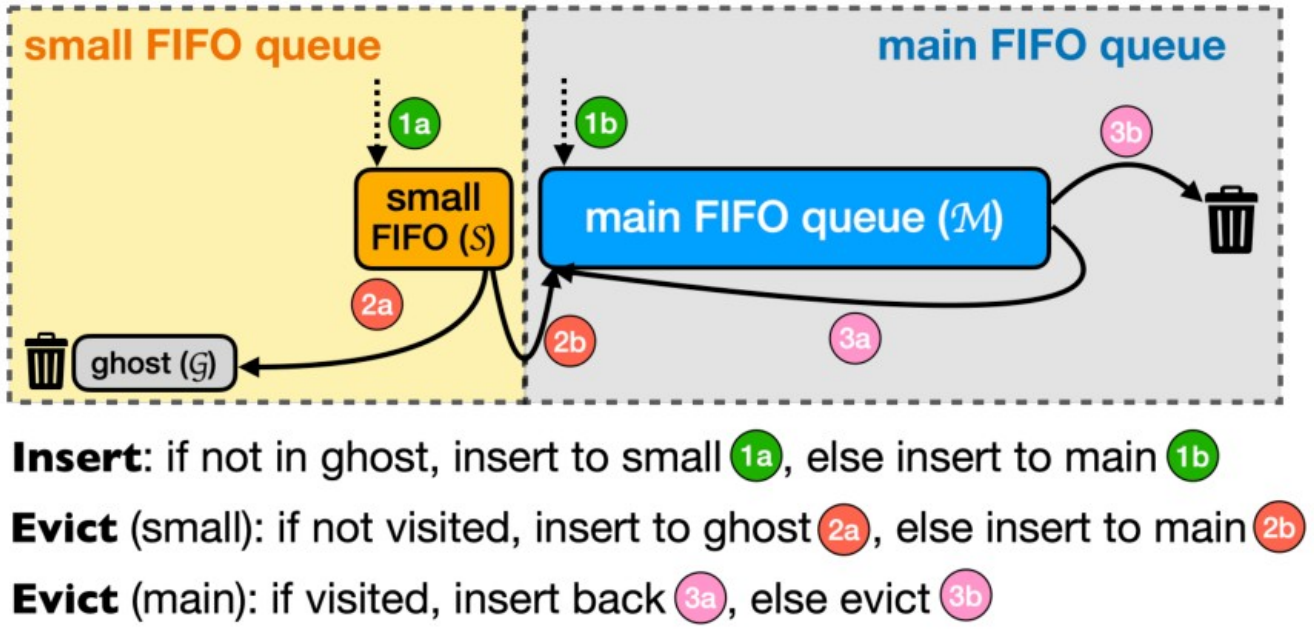
S **Small Queue**

M **Main Queue**

G **Ghost Queue**

Three FIFO Queues, One Cache

S3-FIFO uses Small, Main and Ghost queues together.



Original Figure 5 · Source: Yang et al., SOSP 2023

Small Queue

Filters new objects.

Main Queue

Protects useful objects.

Ghost Queue

Remembers removed IDs.

Source: Original Figure 5 and Section 4.1 · Yang et al., SOSP 2023

The Small Queue (S)

First, the Small Queue uses about 10% of the cache space.



10%

of cache space

New objects usually enter here.

It helps remove objects
that are not used again.

Small filters new objects.

Source: Section 4.1 · Yang et al., SOSP 2023

The Main Queue (M)

Second, the Main Queue uses the remaining 90%.

90%

of cache space

It stores objects that are accessed more frequently.

Objects can reach Main after showing reuse in Small or Ghost.

Main protects useful objects.

Source: Section 4.1 · Yang et al., SOSP 2023

The Ghost Queue (G)

Third, Ghost stores IDs, without the actual data.

It remembers IDs of objects recently removed from Small.

It helps recognize objects that may be useful again.

RECENTLY REMOVED OBJECT IDs



IDs only · No cached payload

Ghost holds as many IDs as Main holds objects. Oldest IDs leave when full.

Source: Section 4.1 · Yang et al., SOSP 2023

Cache Hit: Update the Counter

If an object is already in Small or Main, it is a cache hit.



Its access counter increases, up to a maximum of three.

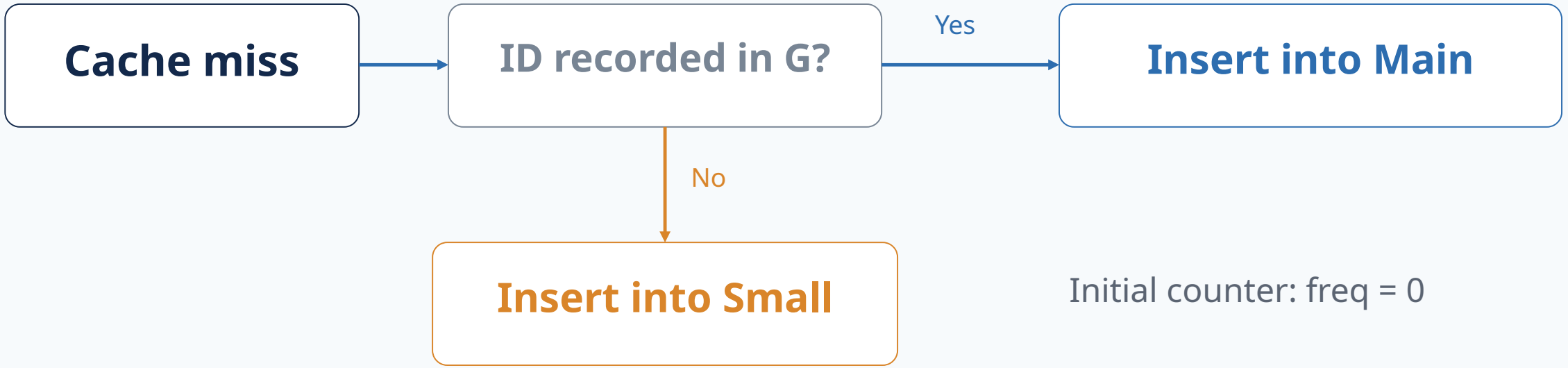
Unlike LRU, S3-FIFO does not move objects after every hit.

Only two bits per object are needed for the counter: 0, 1, 2, 3.

Source: Section 4.1 and Algorithm 1, lines 1–3 · Yang et al., SOSP 2023

Cache Miss: Use Ghost History

If an object is not in Small or Main, it is a cache miss.



Initial counter: freq = 0

If its ID is in Ghost, it enters Main directly.

Otherwise, it enters Small.

Source: Algorithm 1, lines 4–13 · Yang et al., SOSP 2023

Cache Eviction: Which Queue?

Next, let's look at cache eviction.

When the cache is full, S3-FIFO chooses which queue to evict from based on the Small Queue's size.

Small at or above 10%

Evict from Small

Small below 10%

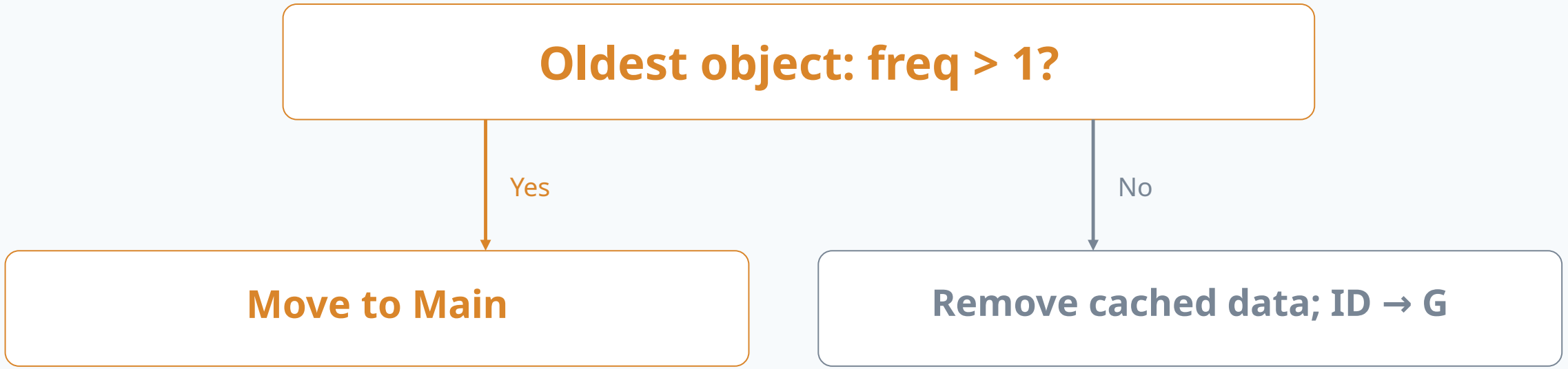
Evict from Main

Each queue checks its oldest object first.

Source: Algorithm 1, lines 14–18 · Yang et al., SOSP 2023

Small Queue Eviction: Filter New Objects

Inspect the oldest object in Small.



Objects with a counter greater than one move to Main.

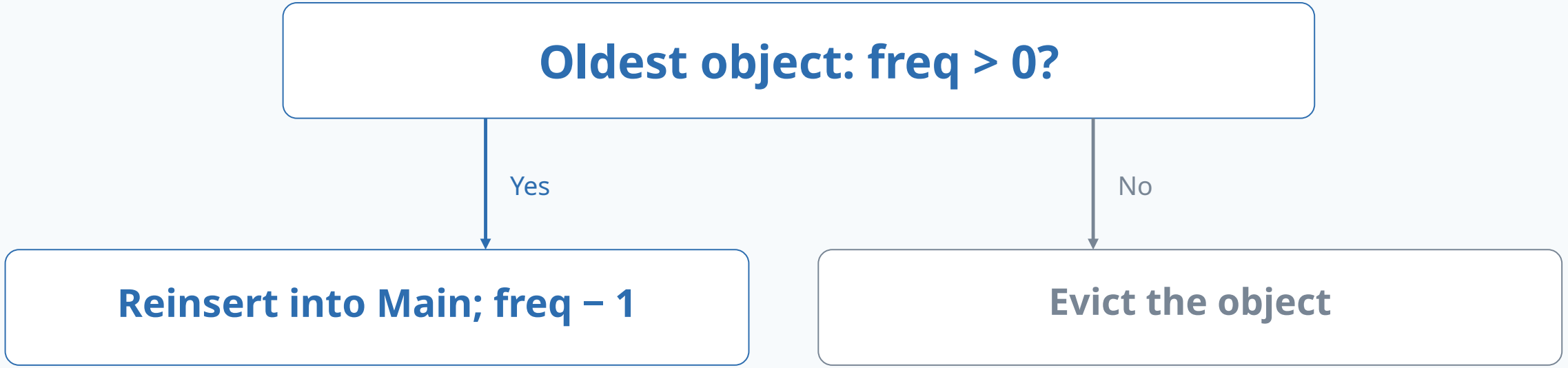
Otherwise, remove the data and send their IDs to Ghost.

The paper clears the access bits when an object leaves Small.

Source: Section 4.1 and Algorithm 1, lines 19–30 · Yang et al., SOSP 2023

Main Queue Eviction: A Second Chance

Inspect the oldest object in Main.



Objects with a positive counter get another chance,

and the counter decreases by one.

Objects with a counter of zero are removed.

Source: Algorithm 1, lines 31–40 · Yang et al., SOSP 2023

Why the Design Is Efficient

The paper combines quick demotion with reinsertion during eviction.

Quick demotion

Small removes objects with little reuse after a short stay.

Lazy promotion

Main reinserts useful objects when considering eviction.

Cache hits only update a small counter.
They do not reorder the queues.

Source: Sections 3.2, 4.1 and 4.3 · Yang et al., SOSP 2023

How the Three Queues Work Together

In summary:

Small filters new objects.

Main protects useful objects.

Ghost remembers removed objects.

This design reduces unnecessary updates and improves cache efficiency.

Now, let's look at the experimental results.

PART 3 · MEMBER 3

Evaluation & Conclusion

What do the paper’s experiments show?

Experimental setup → Efficiency → Scalability

Experimental Setup: Real-World Workloads

The researchers evaluated S3-FIFO using a broad collection of cache traces.

6,594
real cache traces

14
datasets / data groups

WORKLOAD CATEGORIES

- **Storage blocks**
- **Key-value data**
- **Web content**

All experimental results are reported by the original paper's authors.

Source: Table 1 and §5.1, Yang et al., SOSP 2023.

What Did the Researchers Compare?

S3-FIFO was compared with FIFO, LRU and other cache replacement methods.

FIFO

Simple insertion-order eviction

LRU

Evict the least recently used object

S3-FIFO

Three static FIFO queues

Other eviction policies were included in the paper’s evaluation as well.

Compare both cache efficiency and the cost of serving requests.

Source: §§5.1–5.3, Yang et al., SOSP 2023.

What Does Miss Ratio Mean?

A cache miss occurs when the requested data is not in the cache.

Miss ratio = cache misses / total requests

A lower miss ratio is better.

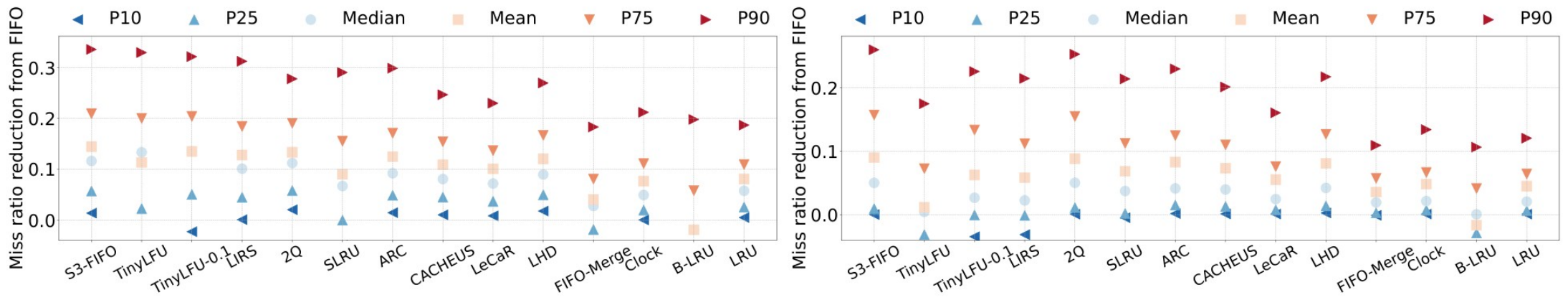
It means more requests can be served directly from the cache.

Miss ratio measures cache efficiency: lower is better.

Source: §5.2, Yang et al., SOSP 2023.

Result 1: Cache Efficiency

Figure 6 compares miss-ratio reduction relative to FIFO, rather than raw miss ratios.



(a) Large cache size, 10% trace footprint

(b) Small cache size, 0.1% trace footprint

Original Figure 6 · Miss-ratio reduction relative to FIFO: a larger reduction is better.

10 / 14

Lowest average miss ratio in 10 of 14 datasets with a relatively large cache.

S3-FIFO performs well across many workloads.

Source: Figure 6 and §5.2, Yang et al., SOSP 2023. The 10/14 dataset result is reported in §5.2 / Figure 7.

What Does Throughput Mean?

Throughput describes how many requests a system can handle in one second.

Throughput = requests handled per second

Higher throughput means more requests can be processed in the same time.

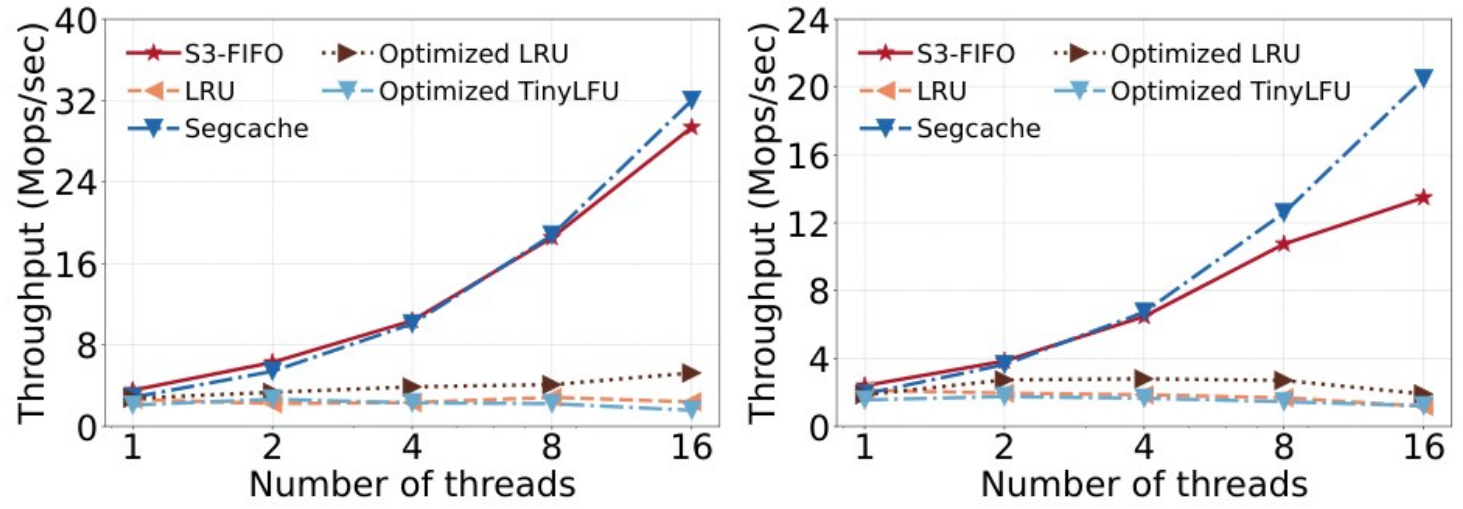
The next figure shows how throughput changes as thread count increases.

Throughput measures processing performance: higher is better.

Source: §5.3, Yang et al., SOSP 2023.

Result 2: Throughput & Scalability

Figure 8 compares throughput as thread count increases.



Original Figure 8 · Synthetic Zipf workload (α = 1.0)

>6×

Compared with optimized LRU at 16 threads.

S3-FIFO scales as the number of threads grows.

Fewer operations on cache hits support better performance under concurrency.

Source: original Figure 8 and §5.3, Yang et al., SOSP 2023. Reported authors' experiments.

A Limitation: The Second Request May Arrive Late

Performance still depends on how users access the data.

Sometimes an object is removed from the Small Queue before its second request arrives.

This can cause an extra cache miss.



Quick demotion is useful, but a late reuse can still cause an additional miss.

Source: §5.2, Yang et al., SOSP 2023.

Final Takeaway

A simple cache algorithm can still deliver strong performance.

Three FIFO queues remove less useful data quickly while keeping useful data for longer.

Lower miss ratios · Better scalability

Thank you for listening.

References

Primary paper, original figures, and the official code and data.

PRIMARY PAPER

Yang, J., Zhang, Y., Qiu, Z., Yue, Y., & Rashmi, K. V. (2023).

FIFO Queues Are All You Need for Cache Eviction. SOSP ’23.

DOI

https://doi.org/10.1145/3600006.3613147

OFFICIAL CODE & DATA

https://github.com/TheSys-lab/sosp23-s3fifo

Figures 1, 5, 6 and 8: Yang et al., SOSP 2023. Mechanism diagrams redrawn from Algorithm 1.